S2S-01: Step 1 — Discover: Migration Scenarios and Inventory

> **Series:** S2S — SaaS to SaaS Migration | **Notebook:** 1 of 9 | **Phase:** Plan | **Step:** Discover | **Created:** March 2026 | **Last Updated:** 10/06/2026

The first step in any SaaS-to-SaaS migration is understanding *why* you are migrating between tenants, inventorying what you have, and confirming what migrates automatically versus what requires manual effort. This notebook guides you through discovery, scenario identification, and tool selection.

> **S2S Migration Journey — 3 Phases / 9 Steps**
>
> **Plan: 1. Discover** | 2. Strategize | 3. Design
>
> **Upgrade:** 4. Prepare | 5. Execute | 6. Integrate
>
> **Run:** 7. Expand | 8. Enable | 9. Optimize

Platform Changes That Affect an S2S Migration

1. **Primary Grail fields and tags enriched at the source** (OneAgent 1.333+) — when redesigning the post-migration tag taxonomy in S2S-03 (Design), prefer OneAgent primary fields and tags (`dt.security_context`, `dt.cost.costcenter`, `dt.cost.product`, `primary_tags.*`) over OpenPipeline parsing where possible. They arrive on every signal without ingest-time processors, and can be set in the same `oneagentctl` call that redirects each agent (S2S-05).
2. **Platform tokens** for new automation in S2S-04/05/10 — this is the right time to retire classic `dt0c01` token use in migration tooling where a platform token or OAuth client is accepted. Use the right scheme: platform tokens go in `Authorization: Bearer …`, classic API tokens in `Authorization: Api-Token …`.
3. **Extensions review** — if either tenant carries custom **Extensions Framework 1.0** extensions, rebuild them as **Extensions 2.0**. Extensions Framework 1.0 reached end of support on 2025-03-31 (its Python 3.8 variant on 2024-10-31); JMX and PMI extensions on Framework 1.0 have their own end-of-support date, **July 1, 2027**.

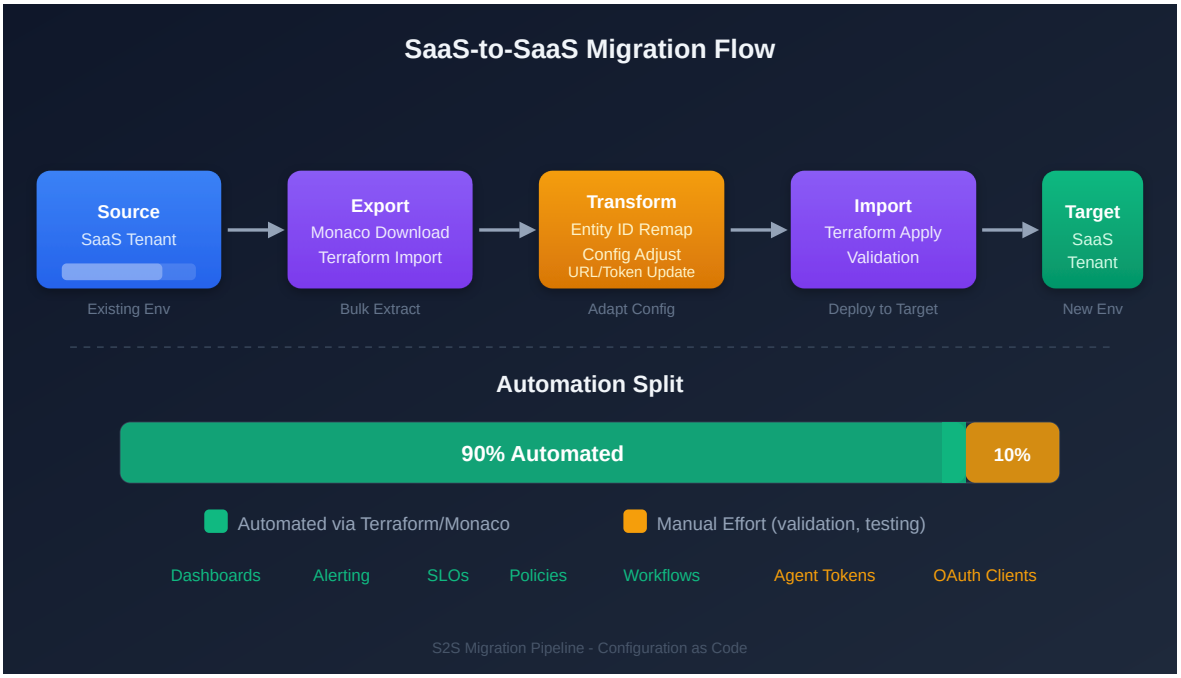
> **Sources:** Primary Grail fields and tags enrichment through OneAgent (DT docs) — *"OneAgent version 1.333"*; End of support announcements (DT docs) — *"EF1 JMX and PMI extensions reach end of support on July 1, 2027."*

Table of Contents

1. Why Migrate Between SaaS Tenants
2. What Migrates and What Does Not
3. Entity Inventory
4. Configuration Inventory
5. Detected Problem Triage and Configuration Debt
6. Migration Tools Comparison
7. The 90/10 Rule
8. Step Completion Checklist

Prerequisites

Requirement	Details
Source Tenant	Active Dynatrace SaaS environment with administrator access
Target Tenant	Provisioned Dynatrace SaaS environment (or planning to provision)
API Tokens	Tokens with `entities.read`, `settings.read` scopes on source tenant
CLI Tools	Monaco CLI v2.x, Terraform v1.5+ (for IAM only)
Stakeholder Access	Ability to document and share findings with decision-makers



1. Why Migrate Between SaaS Tenants

Migrating between Dynatrace SaaS tenants is fundamentally different from migrating from Managed to SaaS. There is no Managed-to-SaaS architecture change, but do not assume the two environments are identical: either one can still be on Dynatrace Classic surfaces (classic cloud integrations, management zones, classic dashboards) or already on Latest Dynatrace, and a SaaS environment can be hosted on a different cloud and region. Record each environment's platform state during discovery rather than assuming parity. S2S also brings its own challenges: entity ID remapping, historical data gaps, and a per-wave overlap while agents move.

Migration Scenarios

Scenario	Description	Example
Tenant Consolidation	Multiple SaaS tenants merged into a single tenant	Merging 5 regional tenants into 1 global tenant after M&A
Account Restructuring	Redistribute environments across different account structures	Spinning off a business unit into its own Dynatrace account
Regional Relocation	Move to a different SaaS region for compliance or performance	Moving from US-hosted to EU-hosted SaaS cluster for GDPR

Scenario	Description	Example
License Restructuring	Restructure DPS allocation across tenants	Moving from multiple small tenants to a single enterprise agreement
Hosting-Cloud Change	Move the Dynatrace SaaS environment itself to a cluster on a different cloud provider	An AWS-hosted environment replaced by an Azure-hosted one
Workload Cloud Change	The *monitored* workloads move to a different cloud provider	Applications rebuilt from AWS onto Azure, with new cloud connections
Environment Promotion	Promote a staging or POC tenant to production	Converting a successful POC into the production monitoring tenant

The Combined Case: Retiring a Cloud

The two cloud rows above are independent decisions, and they are easy to conflate. A **hosting-cloud change** is a tenant move: a new SaaS environment is provisioned on the other cloud and everything in this series applies. A **workload cloud change** is a monitoring change: agents follow the workloads and new cloud connections are created, but it can happen inside one tenant.

When an organization **retires a cloud provider entirely**, both happen at once. Dynatrace documents that *"Data is stored in Amazon Web Services (AWS), Microsoft Azure, or Google Cloud data centers"*, and lists its Azure regions as *"Available on request. Talk to your Dynatrace sales contact."* — so an Azure-hosted target is a commercial conversation before it is a technical one. The consequence for planning is a **dual-cloud window**: the new Azure-hosted environment monitors workloads still running on AWS *and* workloads already moved to Azure, and only once the last AWS workload is gone are the AWS connections and the AWS-hosted source environment retired.

The appendix LAB **S2S-94 (Retiring AWS for Azure)** is the ordered runbook for that case. Two FAQ entries frame it: **FAQ-25** (what carries over to a new tenant — configuration, identity, accumulated state) and **FAQ-17** (the cutover invariants — Go/No-Go gate, parallel-run window, rollback triggers, decommission).

> **Sources:** Data security controls (DT docs) — *"Data is stored in Amazon Web Services (AWS), Microsoft Azure, or Google Cloud data centers."*

> **Key Difference from M2S:** In a Managed-to-SaaS migration, the SaaS Upgrade Assistant handles most of the heavy lifting. For SaaS-to-SaaS, no assistant is documented — you export with Monaco (or Terraform) and import with `monaco deploy`, or optionally through the SaaS Upgrade Assistant as a field practice (**S2S-10** §1).

S2S-Specific Order of Operations

The S2S migration follows an 11-step order of operations within the 9-step framework:

Step	Action	Phase
1	**Assess** — Inventory source tenant	Plan
2	**Provision** — Target SaaS tenant and access (SSO/IAM)	Plan
3	**Export** — Configuration from source tenant	Upgrade
4	**Migrate** — Configuration to target tenant	Upgrade
5	**Rebuild** — Dashboards, SLOs, alerts	Upgrade
6	**Redirect** — OneAgents and operators to target	Upgrade
7	**Reconnect** — Cloud integrations and extensions	Upgrade
8	**Migrate** — Any remaining configuration	Upgrade
9	**Validate** — Data flow and performance	Run
10	**Cutover** — Full switch to target tenant	Run
11	**Decommission** — Source tenant	Run

2. What Migrates and What Does Not

Understanding portability constraints upfront prevents surprises during execution.

Configuration That CAN Be Migrated (with tooling)

Configuration Type	Tool	Notes
Gen3 settings (Settings 2.0)	Monaco, Terraform	Bulk export/import via `monaco download`
Dashboards and notebooks	Monaco (documents type), Terraform	Entity ID references must be updated in target
Workflows and automations	Monaco (automations type), Terraform	Requires OAuth client credentials
OpenPipeline rules	Monaco (openpipeline type), Terraform	Bucket references must match target
Grail bucket configuration	Monaco (bucket type), Terraform	Retention policies may differ
SLO definitions	Monaco (settings/slo-v2 type), Terraform	Metric expressions may reference entity IDs
Management zones	Monaco, Terraform	Zone rules and entity assignments
Auto-tagging rules	Monaco, Terraform	Tag rules and conditions
Service detection rules	Monaco, Terraform	Custom service naming and merging
Request attributes	Monaco, Terraform	Capture rules for request metadata
Synthetic monitors	Monaco, Terraform	Requires classic API token (v1.88.0+)
Segments	Monaco, Terraform	Download, deploy, and delete
Classic dashboards	Monaco, Terraform	JSON export/import; entity IDs must be updated

What CANNOT Be Migrated

Item	Reason	Impact
Historical metrics, logs, traces	Stored in source tenant's Grail	Run parallel tenants during transition

Item	Reason	Impact
Entity IDs	Unique per tenant, auto-generated	Remap references in dashboards/SLOs
Dynatrace Intelligence baselines	Learned from source data	Relearned from target data — measure each host's history depth instead of assuming a fixed period (**FAQ-25** §5)
Session replay recordings	Bound to source tenant	Accept gap or extend parallel period
Problem history	Stored in source tenant	Export key problems as documentation
Credential Vault secrets	Security — secrets cannot be exported	Recreate in target Credentials Vault
API tokens	Security — environment-specific	Generate new tokens in target
OAuth client secrets	Security — cannot be exported	Create new OAuth clients in target
Synthetic execution history	Bound to source tenant	Execution data starts fresh
Smartscape topology history	Computed per-tenant	Rebuilds automatically in target

> **Important:** Historic data does **not** migrate. Plan for an overlap in which the source stays readable while the target accumulates its own history. A host's OneAgent reports to one tenant at a time, so the overlap is **per wave**, not a period in which both tenants receive the same agent data. Size it from measured history depth in the target (**FAQ-25** §5), not from a fixed number of weeks.

3. Entity Inventory

Run these DQL queries against the **source** tenant to understand the full monitoring footprint. This inventory identifies migration complexity and serves as the validation baseline after cutover.

> **Compare like with like.** The three host surfaces do not agree. Over the same 24 hours on a validation tenant (10/01/2026), `smartscapeNodes "HOST"` returned **7** hosts, `fetch dt.entity.host` returned **33**, and the billing usage events named **39**. FAQ-16 explains why Smartscape and the classic entity store differ. Pick one surface, use it in both the source and the target, and run the same query on both. For license sizing, read host consumption from the billing usage events (ADOPT-02 § 6) rather than from either entity count.

Host Inventory

```
// Host inventory by cloud provider and OS (Smartscape)
// cloud.provider is aws / azure / gcp on cloud-hosted hosts and null
otherwise.
// from:-7d includes hosts that reported at any time in the last week, not
only in the
// default window (see the stale-node rule in the DQL syntax table).
smartscapeNodes "HOST", from:-7d
| fieldsAdd provider = coalesce(cloud.provider, "none (on-premises or
undetected)")
| summarize hosts = count(), by:{provider, os.type}
| sort hosts desc

// Classic fallback: fetch dt.entity.host reads the classic entity store,
which can retain
// hosts Smartscape no longer lists (validation tenant, 09/28/2026: 11 classic
vs 9 Smartscape
// hosts). Its cloud-tag fields (awsNameTag / azureResourceGroupName) do not
exist on the
// Smartscape node, so the tag-presence if-chain the earlier version used is
not portable.
```

Kubernetes Cluster Inventory

```
// Kubernetes cluster inventory (Smartscape)
smartscapeNodes "K8S_CLUSTER", from:-7d
| fields name, id
| sort name asc

// Classic fallback: fetch dt.entity.kubernetes_cluster | fields entity.name,
id
// It reads the classic entity store, which can list clusters Smartscape no
longer does —
// cross-check if the two counts disagree.
```

Service Inventory

```
// Service inventory (Smartscape)
smartscapeNodes "SERVICE", from:-7d
| summarize services = count()

// Technology breakdown: the SERVICE node carries no populated service-type
field — on the
// validation tenant dt.service.sdv1_type was null for all 38 services
(09/28/2026). For a
// by-technology breakdown, use the classic fallback:
//   fetch dt.entity.service | summarize count = count(), by:{serviceType}
```

Application and Synthetic Inventory

```
// Application and synthetic monitor counts
smartscapeNodes "FRONTEND"
| filter frontend.type == "web"
| summarize app_count = count()
| append [smartscapeNodes "BROWSER_MONITOR" | summarize browser_monitor_count
= count()]
| append [smartscapeNodes "HTTP_MONITOR" | summarize http_monitor_count =
count()]
| append [smartscapeNodes "NETWORK_AVAILABILITY_MONITOR" | summarize
network_monitor_count = count()]

// Smartscape (preferred, verified 07/2026): dt.entity.application maps to the
FRONTEND node,
// filtered on frontend.type == "web" (mobile apps are the same node with
frontend.type ==
// "mobile"). This corrects an earlier note here that claimed no Smartscape
equivalent existed.
// FRONTEND also carries id_classic holding the APPLICATION-* id, for joining
migrated and
// unmigrated queries. Unlike ActiveGate, `fetch dt.entity.application` does
still work and remains
// a genuine fallback — but it reads the classic entity store, which can
retain entities Smartscape
// (live topology) no longer lists, so cross-check if the two counts disagree.
// Smartscape (preferred, verified 07/2026): dt.entity.synthetic_test maps to
the BROWSER_MONITOR
// node (individual steps are a separate BROWSER_MONITOR_STEP node). HTTP
monitors are HTTP_MONITOR,
// multi-protocol monitors NETWORK_AVAILABILITY_MONITOR, private locations
SYNTHETIC_LOCATION. This
// corrects an earlier note here that claimed no Smartscape equivalent
existed. Unlike ActiveGate,
// `fetch dt.entity.synthetic_test` does still work and remains a genuine
fallback — it reads the
// classic entity store, which can retain entities Smartscape (live topology)
no longer lists.
// Add the three monitor counts together for the "Synthetic Tests" row —
browser monitors alone
// undercount it.
```

ActiveGate Inventory

```
// ActiveGate inventory
smartscapeNodes "ACTIVEGATE"
| summarize activeGateCount = count()

// Correction (verified 07/2026): this cell previously ran `fetch
dt.entity.active_gate` and
// carried a note claiming Smartscape had no ActiveGate node. Both were wrong.
There is NO classic
// ActiveGate entity type in any spelling (active_gate,
environment_active_gate,
// environment_activegate), so the classic query returned zero rows in every
tenant —
// indistinguishable from "no ActiveGates deployed". `smartscapeNodes
"ACTIVEGATE"` (no
// underscore) is the working path, and it works on tenants today.
// Field maps: entity.name → name, softwareVersion → dt.active_gate.version,
// networkZone → dt.network_zone.id.
// This cell previously fell back to counting `fetch dt.entity.host`, which
returns the HOST count,
// not the ActiveGate count — a workaround for the missing entity type that
silently reported the
// wrong number. The ACTIVEGATE node makes the workaround unnecessary.
// ActiveGate 1.343 (published 07/15/2026, staged tenant rollout from
07/28/2026) deprecates
// GET /api/v2/activeGates, /api/v2/activeGates/{agId} and
/api/v2/activeGates/groups in favour of
// this same Smartscape node — the classic entity and the classic REST
endpoints were retired as one
// move. If you need a REST surface in the meantime, the classic Entities API
v2 selector
// (GET /api/v2/entities?entitySelector=type("ENVIRONMENT_ACTIVE_GATE")) is a
different surface and
// may still respond; the DQL `fetch dt.entity.*` form does not.
```

Configuration Change Audit

Understanding what has been actively changed in the last 30 days helps prioritize which configurations are actively managed versus stale.

```
// Audit log: recent Settings 2.0 configuration changes by schema (last 30
days)
// Data object corrected 09/24/2026. The Dynatrace audit trail is NOT in
`logs`: the former
// `fetch logs | filter matchesPhrase(log.source, "audit")` matched nothing,
or matched an
// unrelated file-based audit log (a database .aud file on the validation
tenant). Environment
// audit records are structured events in `dt.system.events` with event.kind
== "AUDIT_EVENT".
// Settings changes carry the schema in details.dt.settings.schema_id.
fetch dt.system.events, from:-30d
| filter event.kind == "AUDIT_EVENT" and event.provider == "SETTINGS"
| filter in(event.type, {"CREATE", "UPDATE", "DELETE"})
| summarize changes = count(), by:{details.dt.settings.schema_id}
| sort changes desc
| limit 20
```

Entity Summary Table

Record your findings in this table:

Entity Type	Count	Notes
Hosts	___	Include OS and cloud provider breakdown
Kubernetes Clusters	___	Cluster names and versions
Services	___	Include technology breakdown
Applications	___	Web and mobile
Synthetic Tests	___	HTTP, browser, scripted
ActiveGates	___	Environment AGs and cluster AGs
Process Groups	___	Total monitored process groups

4. Configuration Inventory

Beyond entities, you need a count of configuration objects to estimate migration effort. Use the tables below as a checklist — the *Typical Range* column is an illustrative order of magnitude from community practice, not a benchmark.

Gen2 (Classic) Configuration

Category	API	Your Count	Typical Range
Management zones	Config API v1: `/managementZones`	___	5–30
Auto-tags	Config API v1: `/autoTags` or Settings 2.0	___	10–50
Alerting profiles	Config API v1: `/alertingProfiles`	___	5–20
Notification rules	Config API v1: `/notifications` or Settings 2.0	___	5–50
Calculated service metrics	Config API v1: `/calculatedMetrics/service`	___	10–50
Request attributes	Config API v1: `/requestAttributes` or Settings 2.0	___	5–30
Custom services	Config API v1: `/service/customServices`	___	5–20
Application detection rules	Config API v1 or Settings 2.0	___	5–20
Conditional naming rules [1]	Config API v1 or Settings 2.0	___	5–15
Maintenance windows	Config API v1 or Settings 2.0	___	3–10
Classic dashboards	Dashboard API v1	___	20–200
Credential vault entries	Environment API v2: `/credentials` (secrets are never returned)	___	5–20

[1] Classic only. Service naming upgrade guide (DT docs): *"In Latest Dynatrace with Smartscape on Grail, service naming rules (builtin:naming.services) no longer apply."* Inventory them to migrate a Classic target; don't plan to recreate them on a Latest Dynatrace one.

Gen3 (Grail) Configuration

Category	API	Your Count	Typical Range
Settings 2.0 schemas (total)	Settings API	___	50–500
Dashboards (modern)	Document API	___	20–200
Notebooks	Document API	___	5–50
SLO definitions	SLO API / slo-v2	___	10–100
Synthetic monitors	Synthetic API	___	10–100
Workflows	Automation API	___	5–30
OpenPipeline rules	OpenPipeline API	___	1–10
Grail buckets	Bucket API	___	1–10
Segments	Segment API	___	1–20
K8s enrichment rules	Settings API: `builtin:kubernetes.generic.metadata.enrichment`	___	1–20

5. Detected Problem Triage and Configuration Debt

Discovery is not just about counting what you have — it is also about identifying what you should **not** migrate. Active detected problems and stale configuration carry over to the target tenant if not triaged, creating noise that obscures real issues during the critical parallel operation period.

Detected Problem Inventory

Run these queries against the source tenant to understand the current problem landscape. Large numbers of active, short-lived or duplicate problems indicate configuration debt that should be resolved *before* migration — not after.

Signal	Action
Active problems > 500	Triage before migration — most are likely noise or duplicates
A large share of problems closing within 5 minutes	Tune the detectors behind those types before export (ADOPT-03 § 5)
Problems older than 30 days	Investigate — likely stale or auto-resolved but not closed

Configuration Debt Cleanup Opportunity

Migration is the best time to leave legacy behind. Stale configuration inflates the export, slows Monaco deploy, and creates confusion in the target tenant.

Candidate	How to Identify	Recommendation
Disabled notification rules	`settings.read` audit — any notification with `enabled: false`	Do not migrate
Inactive synthetic monitors	Monitors with no executions in 30+ days	Triage — migrate only active monitors
Stale maintenance windows	Windows with past end dates or no recurrence	Do not migrate
Unused management zones	Zones with zero entity matches	Do not migrate
Legacy auto-tag rules	Tags that duplicate OneAgent attribute enrichment	Replace with primary tags at agent level

> **Lesson from real migrations:** One engagement discovered 366 maintenance windows in the source tenant — all with expired dates. Migrating them would have

> cluttered the target tenant with useless configuration. Triaging before export saved significant cleanup effort later.

```
fetch dt.davis.problems, from:-30d
| summarize {
    total = count(),
    active = countIf(event.status == "ACTIVE"),
    closed_within_5min = countIf(resolved_problem_duration &lt; 5m),
    duplicate = countIf(dt.davis.is_duplicate == true)
  }
| fieldsAdd triage_recommendation = if(active &gt; 500,
    then: "TRIAGE BEFORE MIGRATION — tune short-lived and duplicate sources",
    else: "Manageable — review active problems during parallel period")

// The frequent-issue flag is not used here: frequent issue detection is being
phased out and
// was set on 0 of 15,227 problems on the validation tenant (ADOPT-03 § 5).
```

6. Migration Tools Comparison

Tool	Best For	Strengths	Limitations
Monaco	Bulk config export/import: settings, documents, automations, buckets, segments, slo-v2, openpipeline, classic API; account IAM through `monaco account`	`monaco download` bulk export, `monaco deploy` with automatic dependency resolution, no HCL knowledge needed	No state management, no drift detection; account commands need an OAuth client
Terraform	IAM (policies, groups, bindings), ongoing infrastructure-as-code management	State tracking, drift detection via `terraform plan`, cross-platform resource references, bulk export with the provider's `-export` utility	Requires HCL knowledge; some resources are excluded from a default export and must be named
Settings API	Targeted, surgical changes to specific settings	Fine-grained programmatic control	Custom scripting required for large-scale migration

> **Note:** The SaaS Upgrade Assistant is documented for a Managed source only — *"SaaS Upgrade Assistant imports your Dynatrace Managed environment configuration"* (SaaS Upgrade Assistant (DT docs)). No SaaS-source path is documented, so this series deploys with Monaco and Terraform directly (**S2S-10**). Uploading a packaged Monaco export to the assistant on a SaaS target is a field practice — S2S-10 builds the package as an option; rehearse it before relying on it.

Where the Two Tools Differ

- **Account IAM** (groups, policies, boundaries, users) — both can do it. Monaco uses dedicated `monaco account download` / `monaco account deploy` commands with an OAuth client; Terraform uses the `dynatrace_iam_*` resources.
- **State and drift** — Terraform keeps state and `terraform plan` shows drift; Monaco deploys are stateless, so drift checking is a download-and-diff you script yourself.

Choose based on your team's existing expertise. A common split, and the one this series follows, is Monaco for bulk environment configuration and Terraform for account IAM, because IAM is the part teams most want under state.

> **Sources:** Monaco account configuration (DT docs) — *"Using Monaco, you can define users, service users, groups, policies, and boundaries as dedicated types in YAML configuration files."*; *"Account management requires OAuth credentials."*

Recommended Approach

Use **Monaco for bulk configuration** and **Terraform (or `monaco account`) for IAM**:

```
# Step 1: Monaco download from source tenant
monaco download --manifest manifest.yaml --environment source-tenant

# Step 2: Update manifest to point at target tenant
# Step 3: Validate and deploy
monaco deploy manifest.yaml --environment target-tenant --dry-run
monaco deploy manifest.yaml --environment target-tenant

# Step 4: Use Terraform only for IAM
terraform apply -target=dynatrace_iam_policy.example
```

> **Download limitations:** Cloud provider credentials (AWS, Azure, K8s) can be deployed by Monaco but **not exported** via `monaco download`. These must be recreated manually in the target tenant regardless of tool choice.

7. The 90/10 Rule

In community practice, migration teams describe the same shape every time — treat the numbers as a planning heuristic, not a measured ratio:

Phase	Effort	What It Covers
Tooled export/import (most of the configuration)	A small share of total effort	Settings 2.0, dashboards, SLOs, notification rules, enrichment rules, OpenPipeline
Manual remediation (a small remainder)	Most of the effort	Entity ID remapping, webhook URL updates, IAM redesign, cloud integration reconfiguration, parallel validation

Why Manual Effort Dominates

- **Entity IDs change** between tenants — every dashboard filter, SLO metric expression, and notification rule that references an entity ID must be updated
- **Integrations are tenant-specific** — webhook URLs, cloud provider connections, and SSO configurations must be reconfigured
- **Historical data cannot move** — parallel operation is required to maintain continuity
- **Dynatrace Intelligence must relearn** — baselines rebuild from target data; measure per-host history depth rather than assuming a fixed period (**FAQ-25** §5)

Items That Require Manual Attention

Item	Why It Cannot Be Automated
Entity ID references in dashboards	IDs are auto-generated per tenant

Item	Why It Cannot Be Automated
Entity ID references in SLO expressions	Metric selectors embed entity IDs
Webhook notification URLs	Network paths differ between tenants
Cloud provider credentials	Secrets cannot be exported
SSO/SAML configuration	Identity provider settings are tenant-specific
Synthetic private locations	ActiveGate-bound, environment-specific
Extensions 2.0	Monaco does not export extension installations; Terraform can install/activate (`dynatrace_hub_extension_active_version`) and configure (`dynatrace_hub_extension_v2_config`) Hub extensions — otherwise reinstall from Hub

8. Step Completion Checklist

Before proceeding to **Step 2 — Strategize**, confirm that you have completed each item:

Checkpoint	Status
Migration scenario identified (consolidation, split, regional relocation, hosting-cloud change, workload cloud change — or both, when retiring a cloud)	[ ]
Platform state of source and target recorded (hosting cloud and region; Classic vs Latest surfaces in use)	[ ]
Entity inventory complete (hosts, services, K8s clusters, applications, synthetics, ActiveGates)	[ ]
Configuration inventory complete (Gen2 counts + Gen3 counts)	[ ]
detected problem triage complete — frequent/duplicate events suppressed or tuned	[ ]

Checkpoint	Status
Configuration debt identified — stale maintenance windows, disabled rules, inactive monitors cataloged	[ ]
Non-portable items identified (credentials, tokens, entity IDs, historical data)	[ ]
Migration tools selected (Monaco for bulk config + Terraform for IAM)	[ ]
90/10 manual items documented	[ ]
Stakeholders informed of historical data limitation and parallel-run requirement	[ ]

Next Step

> **S2S-02: Step 2 — Strategize** — Define your migration approach, timeline, and risk assessment. Choose between big-bang and phased migration, plan your parallel-run period, and identify dependencies and risks.

Additional Resources

- Monaco Configuration as Code
- Dynatrace Terraform Provider
- Settings API
- Grail Data Lakehouse
- DQL Reference

Summary

In Step 1, you:

- Identified your migration scenario (consolidation, split, regional relocation, hosting-cloud change, workload cloud change, or environment promotion)
- Documented what configuration migrates with tooling versus what requires manual recreation

- Completed an entity inventory (hosts, services, K8s clusters, applications, synthetics, ActiveGates)
- Completed a configuration inventory (Gen2 classic + Gen3 Grail counts)
- Selected migration tools (Monaco for bulk config, Terraform for IAM)
- Understood the 90/10 shape: most configuration moves with tooling, but the small remainder takes most of the effort

---

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.